

Spark SQL Catalyst

核心架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块: Catalyst (sql/catalyst)

版本: v3.3.1

生成工具: AI Architecture Analyzer

生成日期: 2026年03月25日

第1章 模块概述

1.1 模块定位

Catalyst 是 Apache Spark SQL 的查询编译核心引擎，位于 sql/catalyst 子模块中。它负责将用户提交的 SQL 语句或 DataFrame/Dataset API 调用转换为优化后的逻辑计划（LogicalPlan），是 Spark SQL 整个查询处理管线中最核心的组件。Catalyst 采用基于规则（Rule-based）和基于代价（Cost-based）的混合优化策略，通过树节点变换框架实现灵活可扩展的查询优化。

Catalyst 模块不依赖具体的执行引擎（如 Spark Core 的 RDD），它是一个纯粹的查询编译框架，处理 SQL 解析、语义分析、逻辑优化等阶段，输出优化后的逻辑计划供上层的 SparkPlanner 转换为物理执行计划。

1.2 核心目标

- SQL 解析: 将 SQL 文本通过 ANTLR4 解析为未解析的逻辑计划树（Unresolved LogicalPlan）
- 语义分析: 通过 Analyzer 解析列名、函数名、类型推断等，生成已解析的逻辑计划
- 逻辑优化: 通过 Optimizer 应用数十条优化规则（谓词下推、常量折叠、连接重排序等）
- 物理计划策略: 提供 QueryPlanner 抽象，将逻辑计划映射为物理计划策略
- 代码生成: 通过 Whole-Stage CodeGen 框架将表达式编译为高效的 Java 字节码
- 元数据管理: 通过 SessionCatalog 管理数据库、表、函数等元数据信息

1.3 模块规模统计

Catalyst 模块总计约 598 个 Scala 源文件，分布在 19 个核心包中，是 Spark SQL 中代码量最大、复杂度最高的子模块。

包名	文件数	核心职责
analysis/	~200	语义分析：名称解析、类型强制转换、函数解析
expressions/	~178	表达式体系：所有 SQL 表达式定义与求值
plans/	~48	查询计划：逻辑计划和物理计划节点定义
optimizer/	~43	查询优化：数十条逻辑优化规则
util/	~41	工具类：日期时间、HLL++、字符串等
catalog/	~16	元数据目录：SessionCatalog、ExternalCatalog
parser/	9	SQL 解析：ANTLR4 语法树到逻辑计划
rules/	5	规则引擎：Rule + RuleExecutor 框架

包名	文件数	核心职责
trees/	4	树节点框架：TreeNode 基础设施
encoders/	4	编码器：JVM 对象与 InternalRow 转换
planning/	3	计划策略：QueryPlanner 框架
csv/json/xml/	~21	结构化数据格式：解析/生成/Schema推断
streaming/	4	流式处理：流式逻辑计划节点

第2章 核心架构设计

2.1 整体分层架构

Catalyst 的架构可以分为四个核心层次，对应 SQL 查询处理的完整管线：

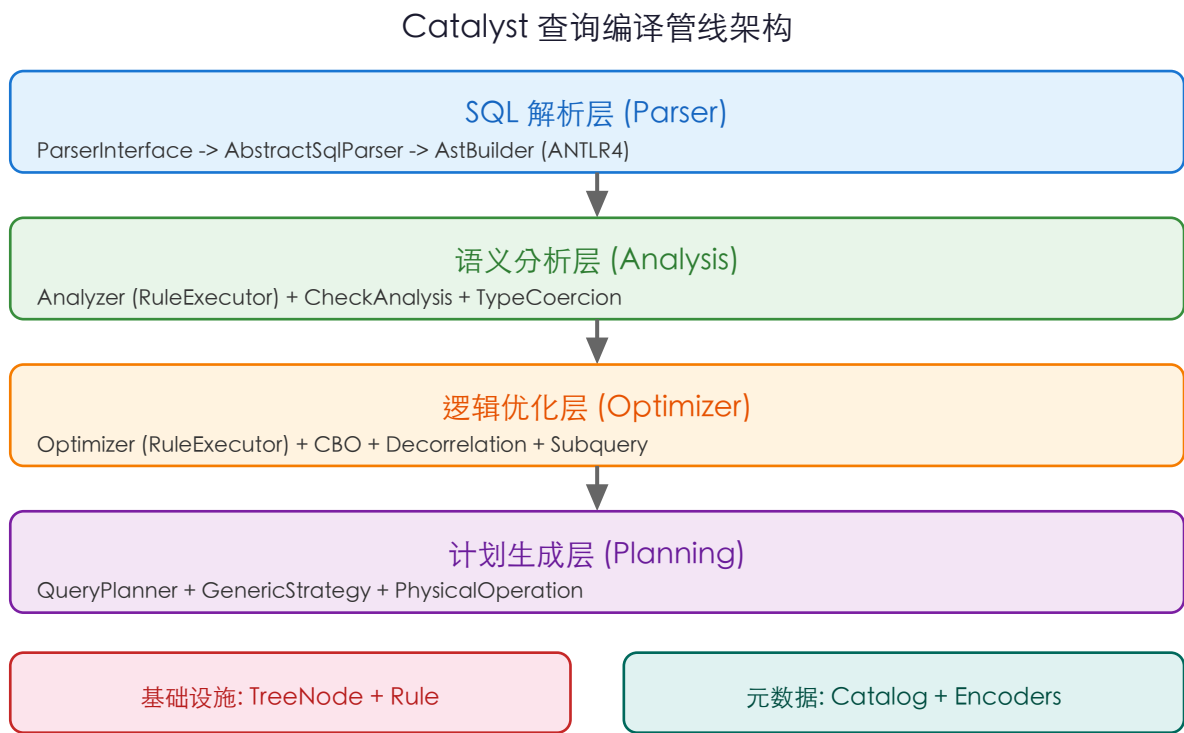


图2-1 Catalyst 整体分层架构图

2.2 核心类继承体系

Catalyst 所有的查询计划节点和表达式节点都继承自统一的 `TreeNode` 基类，形成了一棵递归定义的树结构。这是整个 Catalyst 框架的核心基石：

```
TreeNode[BaseType] // ■■■■■■
+-- Expression // ■■■■■■
| +-- LeafExpression // ■■■■■■Literal, Attribute■
| +-- UnaryExpression // ■■■■■■Not, IsNull, Cast■
| +-- BinaryExpression // ■■■■■■Add, And, EqualTo■
| +-- TernaryExpression // ■■■■■■If, Substring■
| +-- AggregateFunction // ■■■■■■Sum, Count, Avg■
| +-- NamedExpression // ■■■■■■Alias, AttributeReference■
| +-- SubqueryExpression // ■■■■■■
+-- QueryPlan[PlanType] // ■■■■■■
+-- LogicalPlan // ■■■■■■
| +-- LeafNode // ■■■■■■LocalRelation, Range■
| +-- UnaryNode // ■■■■■■Filter, Project, Sort■
| +-- BinaryNode // ■■■■■■Join, Union■
| +-- Command // DDL ■■
+-- SparkPlan (sql/core) // ■■■■■■Catalyst ■■■■
```

2.3 规则引擎框架

Catalyst 的核心优化机制建立在 `Rule` + `RuleExecutor` 框架之上。`Rule` 是一个将树节点变换为新树节点的函数，`RuleExecutor` 按 `Batch` 组织多条 `Rule`，以 `FixedPoint`（迭代到收敛）或 `Once`（执行一次）策略执行。

```
abstract class Rule[TreeType] {
  def apply(plan: TreeType): TreeType // ■■■■■■■■■■
  val ruleName: String // ■■■■
}

abstract class RuleExecutor[TreeType] {
  case class Batch(name: String, strategy: Strategy, rules: Rule*)
  case object Once extends Strategy // ■■■■
  case class FixedPoint(maxIter: Int) // ■■■■■■
  def execute(plan: TreeType): TreeType // ■■■■■■ Batch
}
```

`Analyzer` 和 `Optimizer` 都是 `RuleExecutor[LogicalPlan]` 的子类，它们各自定义了不同的规则批次（Batch）来完成各阶段的变换。

第3章 核心类设计模式详解

3.1 TreeNode — Composite（组合）模式

TreeNode 是 Catalyst 中最核心的基类，它采用典型的组合模式将单个节点和复合节点统一对待。每个 TreeNode 持有 children 列表，并提供 transform/foreach/collect 等递归遍历方法。

- TreeNode.children: 返回子节点列表，叶子节点返回空列表
- TreeNode.transform(rule): 递归地对树进行自底向上/自顶向下变换
- TreeNode.foreach(f): 递归遍历所有节点执行给定函数
- TreeNode.collect[B](pf): 递归收集满足条件的节点
- TreeNode.makeCopy(args): 基于 Product trait 反射创建节点副本

设计意图：通过组合模式统一处理 Expression 树和 QueryPlan 树，使得同一套 Rule 机制可以无差别地作用于表达式变换和计划变换。TreePatternBits 位图优化可快速跳过不匹配的规则，避免无效遍历。

3.2 Rule / RuleExecutor — Strategy（策略）+ Chain of Responsibility（责任链）模式

Rule 接口定义了 apply(plan) 方法，每条 Rule 是一个独立的变换策略。RuleExecutor 将多条 Rule 组织为 Batch，按策略串行执行，形成责任链。

- Strategy 模式体现: Once/FixedPoint 策略决定执行方式
- 责任链模式体现: 多条 Rule 按顺序 foldLeft 依次处理计划树
- RuleExecutor.execute(plan): 遍历每个 Batch，在 Batch 内 foldLeft 应用 Rules

设计意图：解耦优化规则的定义与执行策略，允许灵活添加/移除/排序规则，支持通过配置排除特定规则（excludedRules 机制）。

3.3 Analyzer — Template Method（模板方法）模式

Analyzer 继承 RuleExecutor[LogicalPlan]，通过 batches 方法定义了语义分析的完整规则序列。子类（如 SparkAnalyzer）可以覆盖 extendedResolutionRules 等钩子方法来注入自定义分析规则，这是典型的模板方法模式。

- batches: 定义固定的分析批序列（模板）
- extendedResolutionRules: 扩展点，子类可插入自定义规则
- postHocResolutionRules: 后置分析规则扩展点

3.4 Optimizer — Template Method + Builder 模式

Optimizer 同样继承 RuleExecutor[LogicalPlan]，通过 defaultBatches 定义默认优化规则集，flattenBatches 方法支持灵活组合 Batch 序列。子类 SparkOptimizer 可通过 extendedOperatorOptimizationRules 等方法扩展优化规则。

- defaultBatches: 定义完整的优化批次（40+ 条规则）
- operatorOptimizationRuleSet: 核心操作优化规则集（可复用于多个 Batch）
- excludedRules / nonExcludableRules: 规则排除机制

3.5 QueryPlanner — Strategy（策略）模式

QueryPlanner 定义了将 LogicalPlan 转换为 PhysicalPlan 的策略框架。它维护一个 GenericStrategy 列表，每个 Strategy 负责处理特定类型的逻辑计划。

- strategies: 策略列表，由子类提供具体实现
- plan(logicalPlan): 迭代所有策略，收集候选物理计划
- planLater(plan): 延迟规划占位符，支持递归规划

设计意图：将逻辑到物理的映射拆分为多个独立策略，每个策略只需关注自己擅长的转换场景。

3.6 SessionCatalog — Facade（外观）模式

SessionCatalog 是会话级元数据管理的外观接口，封装了 ExternalCatalog（Hive Metastore）、FunctionRegistry（内建/临时函数）、GlobalTempViewManager（全局临时视图）等多个子系统。

- ExternalCatalog: 持久化元数据存储（数据库、表、分区）
- FunctionRegistry: 标量函数注册表（500+ 内建函数）
- TableFunctionRegistry: 表函数注册表
- GlobalTempViewManager: 全局临时视图管理

3.7 ExpressionEncoder — Adapter（适配器）模式

ExpressionEncoder 将用户空间的 JVM 对象（case class、Java Bean）与 Spark 内部的 InternalRow 之间进行适配转换，是 Dataset API 的核心组件。

- Serializer: 将 JVM 对象序列化为 InternalRow（通过 Expression 树）
- Deserializer: 将 InternalRow 反序列化为 JVM 对象

3.8 CodeGenerator — Template Method + Factory 模式

CodeGenerator 是代码生成框架的基类，通过 Janino 编译器将 Expression 树编译为 Java 字节码。不同子类（GenerateUnsafeProjection、GenerateOrdering 等）生成不同用途的代码。

- CodegenContext: 代码生成上下文，管理变量声明、import 等
- Expression.genCode(ctx): 每个 Expression 提供自己的代码生成逻辑
- CodegenFallback: 不支持代码生成时的解释执行回退

第4章 核心流程图

4.1 SQL 查询完整编译流程

一条 SQL 语句从输入到生成优化后的逻辑计划，经历以下完整编译流程：

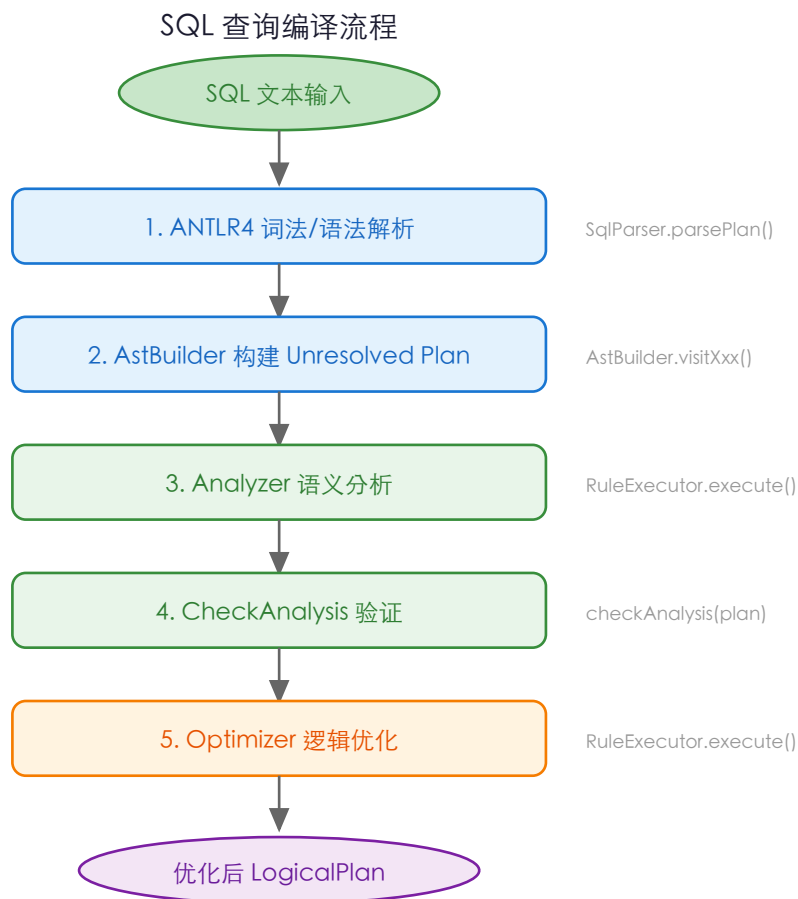


图4-1 SQL 查询完整编译流程图

4.2 Analyzer 语义分析内部流程

Analyzer 是 RuleExecutor 的子类，内部定义了多个 Batch，按顺序依次执行：首先替换 CTE 引用，然后进行多轮迭代解析（名称解析、函数解析、类型强制转换等），最后进行后置处理和验证。

Batch 名称	执行策略	主要规则
Substitution	FixedPoint	CTESubstitution, WindowResolution, ResolveCatalogs
Disable Hints	Once	ResolveHints.DisableHints
Hints	FixedPoint	ResolveHints.ResolveJoinStrategyHints, ResolveHints.ResolveCoalesceHints
Simple Sanity Check	Once	LookupFunctions, CheckDuplicateRelations

Batch 名称	执行策略	主要规则
Resolution	FixedPoint	ResolveRelations, ResolveReferences, ResolveAliases, TypeCoercion, FunctionResolution等
Post-Hoc Resolution	Once	extendedResolutionRules (扩展点)
Cleanup	FixedPoint	CleanupAliases, PullOutNondeterministic

4.3 Optimizer 逻辑优化内部流程

Optimizer 定义了丰富的优化批次，核心分为以下几个阶段：

Batch 名称	策略	核心规则
Finish Analysis	FixedPoint(1)	FinishAnalysis, RewriteWithExpression
Eliminate Distinct	Once	EliminateDistinct
Inline CTE	Once	InlineCTE
Union	FixedPoint	CombineUnions, RemoveNoopOperators
OptimizeOneRowPlan	FixedPoint	OptimizeOneRowPlan, PullupCorrelatedPredicates
Operator Optimization (前)	FixedPoint	PushDownPredicates, ColumnPruning, ConstantFolding, BooleanSimplification等40+条规则
Infer Filters	Once	InferFiltersFromGenerate, InferFiltersFromConstraints
Operator Optimization (后)	FixedPoint	同前,确保推断的Filter也被优化
CBO (Join Reorder)	Once	CostBasedJoinReorder
RewriteSubquery	Once	RewriteCorrelatedScalarSubquery, PullOutNestedData...

第5章 核心时序图

5.1 SQL 解析到逻辑计划时序图

展示一条 SQL 语句从 SparkSession.sql() 到生成优化后 LogicalPlan 的组件交互过程：

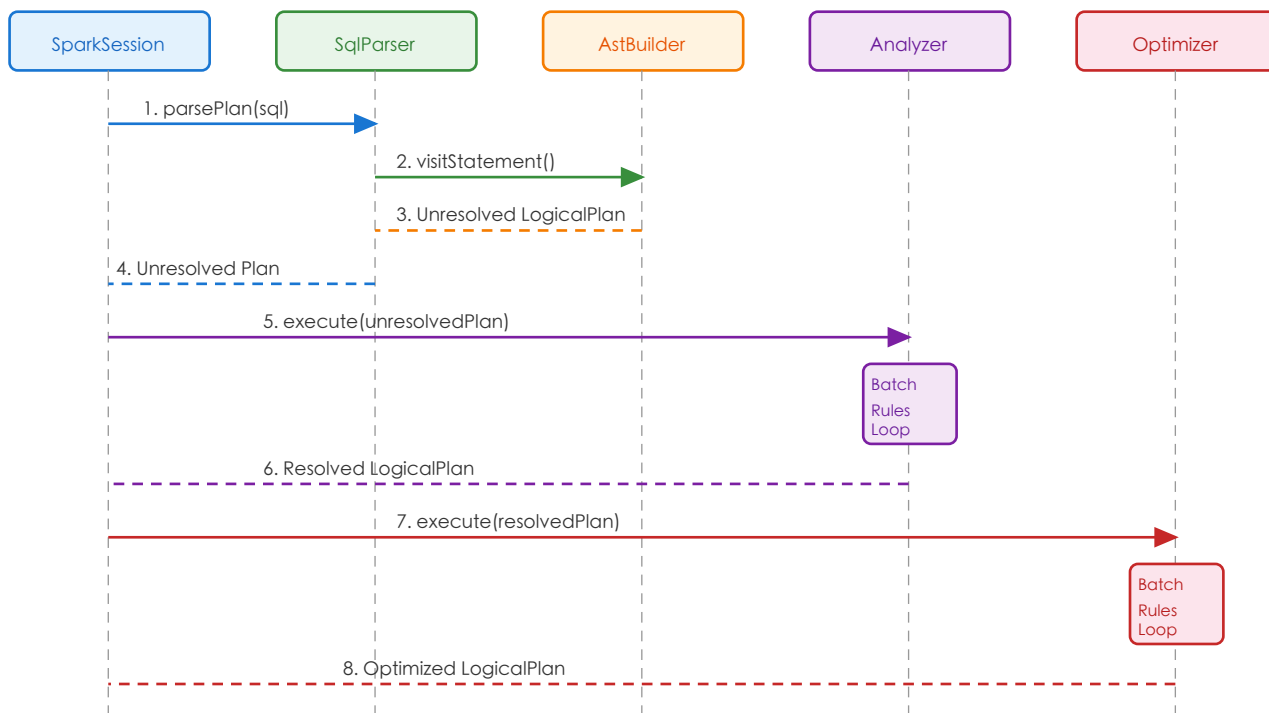


图5-1 SQL 解析到优化逻辑计划时序图

5.2 表达式求值与代码生成时序图

展示 Expression 求值的两条路径——解释执行与代码生成：

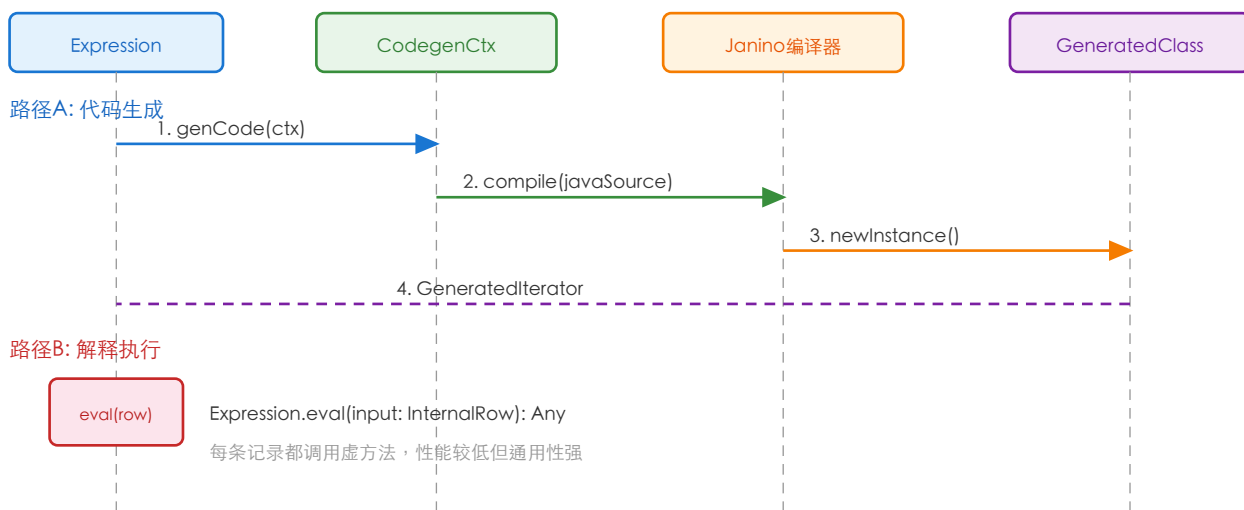


图5-2 表达式求值路径时序图

5.3 Catalog 元数据查询时序图

展示 Analyzer 在解析表名时如何与 SessionCatalog/ExternalCatalog 交互：

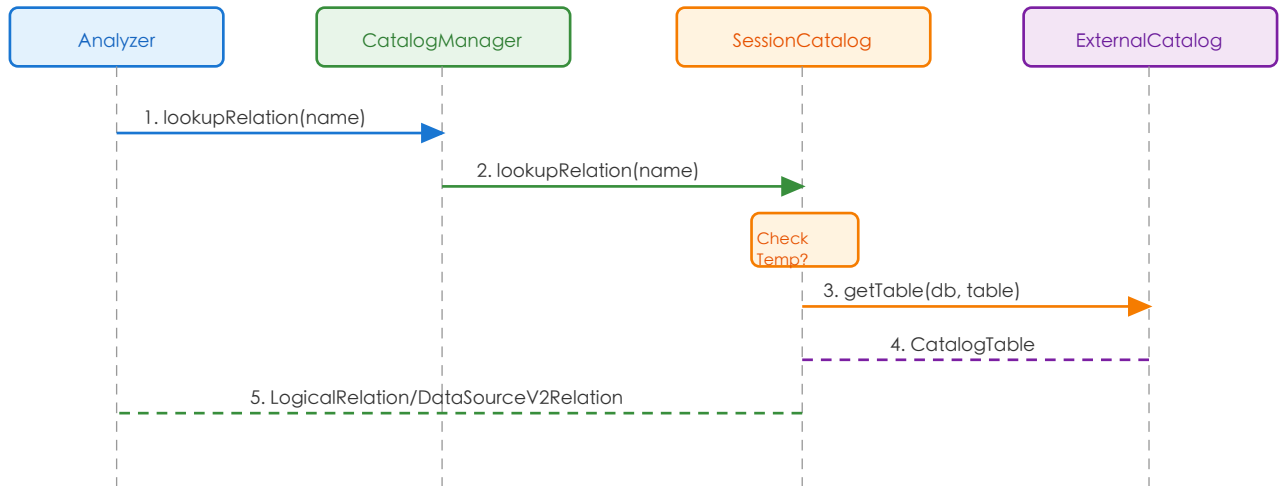


图5-3 Catalog 元数据查询时序图

第6章 关键场景调用链

6.1 场景一: SELECT 查询解析调用链

触发条件：用户执行 `spark.sql("SELECT a, b FROM t WHERE a > 1")`

步骤	类.方法	说明
1	<code>SparkSession.sql(sqlText)</code>	入口：接收 SQL 文本
2	<code>AbstractSqlParser.parsePlan(sqlText)</code>	调用 ANTLR4 解析器
3	<code>AstBuilder.visitSingleStatement(ctx)</code>	访问 AST 根节点
4	<code>AstBuilder.visitQueryPrimary(ctx)</code>	构建 Project + Filter + UnresolvedRelation
5	<code>Analyzer.execute(unresolvedPlan)</code>	启动语义分析
6	<code>ResolveRelations.apply(plan)</code>	解析表名为 <code>LogicalRelation</code>
7	<code>ResolveReferences.apply(plan)</code>	解析列名为 <code>AttributeReference</code>
8	TypeCoercion rules	隐式类型转换（如 <code>Int > Long</code> ）
9	<code>CheckAnalysis.checkAnalysis(plan)</code>	验证所有节点已解析
10	<code>Optimizer.execute(analyzedPlan)</code>	启动逻辑优化
11	<code>PushDownPredicates.apply(plan)</code>	将 WHERE 条件下推至数据源
12	<code>ColumnPruning.apply(plan)</code>	只读取需要的列 a, b

6.2 场景二: JOIN 查询优化调用链

触发条件：执行多表 JOIN 查询，触发基于代价的连接重排序

步骤	类.方法	说明
1	<code>Analyzer.ResolveReferences</code>	解析两个表的列引用
2	<code>NaturalAndUsingJoinResolution</code>	解析 NATURAL/USING JOIN
3	<code>Optimizer.ReorderJoin</code>	基于规则的 JOIN 重排序
4	<code>Optimizer.EliminateOuterJoin</code>	消除不必要的 Outer Join
5	<code>CostBasedJoinReorder.apply(plan)</code>	CBO 连接重排序（基于统计信息）

步骤	类.方法	说明
6	StarSchemaDetection	星型 Schema 检测优化
7	PushDownPredicates	下推 JOIN 条件
8	InferFiltersFromConstraints	从约束推断额外过滤条件

6.3 场景三: 聚合查询处理调用链

触发条件：执行 SELECT col, COUNT(*), SUM(val) FROM t GROUP BY col

步骤	类.方法	说明
1	AstBuilder.visitAggregation(ctx)	构建 Aggregate 逻辑节点
2	FunctionResolution	解析 COUNT/SUM 为 AggregateFunction
3	ResolveReferencesInAggregate	解析聚合中的列引用
4	GroupingAnalyticsTransformer	处理 GROUPING SETS/CUBE/ROLLUP
5	Optimizer.RemoveRedundantAggregates	消除冗余聚合
6	Optimizer.RewriteDistinctAggregates	重写 DISTINCT 聚合
7	PullOutGroupingExpressions	提取分组表达式

6.4 场景四: 子查询去关联调用链

触发条件：执行包含相关子查询的 SQL（如 WHERE EXISTS (SELECT ...)）

步骤	类.方法	说明
1	Analyzer.ResolveSubquery	解析子查询表达式中的引用
2	ValidateSubqueryExpression	验证子查询合法性
3	Optimizer.RewriteCorrelatedScalarSubquery	重写标量相关子查询为 LEFT JOIN
4	DecorrelateInnerQuery.apply(plan)	去关联：提取外部引用转为 JOIN 条件
5	PullOutNestedDataOuterRefExpressions	处理嵌套数据类型的外部引用
6	Optimizer.RewriteNonCorrelatedExists	优化非相关 EXISTS

6.5 场景五: 代码生成调用链

触发条件：Whole-Stage CodeGen 将表达式编译为 Java 字节码

步骤	类.方法	说明
1	Expression.genCode(ctx: CodegenContext)	生成表达式的 Java 代码片段
2	CodegenContext.addReferenceObj()	注册运行时对象引用
3	GenerateUnsafeProjection.create(exprs)	生成 Projection 代码
4	CodeGenerator.compile(code)	使用 Janino 编译 Java 源码
5	ClassBodyEvaluator.cook(javaSource)	Janino 编译并加载字节码
6	GeneratedClass.generate(refs)	实例化生成的类

第7章 配置与扩展

7.1 核心配置参数

配置项	默认值	说明
spark.sql.optimizer.maxIterations	100	优化器最大迭代次数
spark.sql.optimizer.excludedRules	无	排除的优化规则（逗号分隔）
spark.sql.cbo.enabled	false	是否启用基于代价的优化
spark.sql.cbo.joinReorder.enabled	false	是否启用 CBO 连接重排序
spark.sql.codegen.wholeStage	true	是否启用全阶段代码生成
spark.sql.codegen.fallback	true	代码生成失败时是否回退解释执行
spark.sql.planChangeValidation	false	是否验证规则变更合法性
spark.sql.catalogImplementation	in-memory	元数据目录实现(in-memory/hive)
spark.sql.metadataCacheTTLSeconds	无	元数据缓存 TTL

7.2 扩展点

Catalyst 提供了丰富的扩展点，允许用户和下游模块注入自定义逻辑：

- Analyzer.extendedResolutionRules: 注入自定义分析规则
- Analyzer.postHocResolutionRules: 后置分析规则扩展
- Optimizer.extendedOperatorOptimizationRules: 自定义优化规则
- FunctionRegistry: 注册自定义标量函数 (UDF)
- TableFunctionRegistry: 注册自定义表函数 (UDTF)
- ExternalCatalog: 实现自定义元数据存储后端
- GenericStrategy: 实现自定义物理计划生成策略
- ParserInterface: 实现自定义 SQL 方言解析器

第8章 设计亮点与总结

8.1 设计亮点

- 统一的树变换框架: `TreeNode + Rule`
机制将查询分析、优化、验证统一为树到树的变换操作，代码简洁、可扩展性强。仅通过一个 `apply(plan)` 方法即可定义新的优化规则。
- `TreePattern` 位图优化: 每个 `TreeNode` 维护 `TreePatternBits` 位图，标记其包含的节点模式。`Rule` 声明自己关心的模式，不匹配时直接跳过，大幅减少无效遍历。
- 混合优化策略: 同时支持基于规则的优化（RBO，40+ 条规则）和基于代价的优化（CBO，`CostBasedJoinReorder + Statistics`），兼顾效率和质量。
- Whole-Stage CodeGen: 通过 Janino 动态编译，将整个查询管线编译为单个 Java 方法，消除虚函数调用开销，性能接近手写代码。
- 高度可扩展: 通过 `extendedResolutionRules`、`extendedOperatorOptimizationRules` 等钩子，下游模块（Spark SQL Execution、Hive 集成）可以无侵入地注入自定义规则。
- 子查询去关联 (Decorrelation): `DecorrelateInnerQuery` 将相关子查询转换为 JOIN，这是 SQL 查询优化中的经典难题，Catalyst 提供了完整的去关联实现。

8.2 设计限制

- 固定点迭代开销: `FixedPoint`
策略可能导致规则重复执行多轮，当规则数量多或计划树大时，分析/优化阶段耗时可能较长。
- CBO 依赖统计信息: `CostBasedJoinReorder`
需要准确的表统计信息，如果统计信息缺失或过期，CBO 优化效果可能不如 RBO。
- 单体 Analyzer: `Analyzer.scala` 超过 190KB，包含大量嵌套规则类，代码耦合度较高，维护和理解成本较大。

8.3 质量评估

评估维度	评分	说明
代码架构	★★★★★	<code>TreeNode+Rule</code> 框架设计极为优雅，业界标杆
可扩展性	★★★★★	丰富的扩展点，支持无侵入式自定义
设计模式运用	★★★★★	<code>Composite</code> 、 <code>Strategy</code> 、 <code>Template Method</code> 等模式运用纯熟
性能优化	★★★★☆	<code>TreePatternBits</code> 、 <code>CodeGen</code> 优化出色，但 <code>FixedPoint</code> 有改进空间
代码可维护性	★★★☆☆	核心文件过大(<code>Analyzer</code> 190KB, <code>Optimizer</code> 127KB)，建议拆分
文档完整性	★★★★☆	<code>ScalaDoc</code> 注释丰富，但缺少整体架构文档

8.4 总结

Spark SQL Catalyst 是一个设计精良的查询编译框架，其核心的 `TreeNode + Rule + RuleExecutor` 三元组构成了优雅且高效的树变换引擎。通过组合模式统一处理表达式和计划节点、策略模式实现灵活的规则执行、模板方法模式支持可扩展的分析/优化管线，Catalyst 成为了现代大数据 SQL 引擎的设计典范。从 SQL 解析到逻辑优化的完整管线清晰可追踪，约 600 个 Scala 源文件覆盖了表达式系统、查询计划体系、规则引擎、元数据管理、代码生成等核心领域。